AI1220 Assignment 1

Requirements Engineering, Architecture, and Proof of Concept

Collaborative Document Editor with AI Writing Assistant

Course:	AI1220
Semester:	February 2026
Deliverable:	Final written report
Repository:	https://github.com/tabisevi-code/collaborative-editor-ai

Team Member	**Primary Responsibility**
Tabina	Architecture, infrastructure, repository organization
Noel	Backend API, contracts, realtime feasibility
Nurtore	AI design, data model, privacy/cost controls
Ming Ming	Frontend UX, collaboration flows, API usability

This report documents both the intended architecture and the delivered MVP implementation. The final system goes beyond the minimum proof-of-concept by providing live collaboration, async AI jobs, permissions, versioning, export, reproducible developer workflows, and automated tests.

April 3, 2026

Contents

1 Executive Summary **1**

2 Team Responsibilities and Contributions **2**
2.1 Ownership Model . . . 2
2.2 Cross-Cutting Collaboration . . . 2
2.3 Decision-Making Process . . . 3

I Requirements Engineering **4**

3 Requirements Engineering **5**
3.1 Stakeholder Analysis . . . 5
3.2 Functional Requirements . . . 6
3.2.1 Real-Time Collaboration . . . 6
3.2.2 Document Management . . . 8
3.2.3 User Management and Authorization . . . 11
3.2.4 AI Assistant . . . 13
3.3 Non-Functional Requirements . . . 15
3.3.1 Latency and Scalability . . . 15
3.3.2 Security, Availability, and Privacy . . . 15
3.3.3 Usability and Accessibility . . . 15
3.4 User Stories and Scenarios . . . 16
3.5 Requirements Traceability . . . 17

II System Architecture **20**

4 System Architecture **21**
4.1 Architectural Drivers . . . 21
4.2 C4 Level 1 — System Context . . . 21
4.3 C4 Level 2 — Container Diagram . . . 22
4.4 C4 Level 3 — Backend API Component Diagram . . . 23
4.5 Feature Decomposition . . . 24
4.6 AI Integration Design . . . 25
4.6.1 Context and Scope . . . 25
4.6.2 Suggestion UX . . . 26
4.6.3 AI During Collaboration . . . 26
4.6.4 Prompt Design and Provider Strategy . . . 26
4.6.5 Async Job Model . . . 26
4.7 API Design . . . 27
4.7.1 Core API Contracts . . . 27
4.7.2 Authentication and Authorization . . . 29

4.7.3 Communication Model . . . 30
4.8 Code Structure and Repository Organization . . . 30
4.8.1 Top-Level Layout . . . 30
4.8.2 Shared Code and Config . . . 31
4.8.3 Testing Structure . . . 31
4.9 Data Model . . . 31
4.10 Architecture Decision Records . . . 32

III Project Management and Team Collaboration 34

5 Project Management and Team Collaboration 35
5.1 Team Structure and Ownership . . . 35
5.2 Development Workflow . . . 35
5.2.1 Branching Strategy . . . 35
5.2.2 Code Review Process . . . 36
5.2.3 Task Tracking and Communication . . . 36
5.3 Development Methodology . . . 37
5.4 Risk Assessment . . . 37
5.5 Timeline and Milestones . . . 38

IV Proof of Concept and Delivered MVP 39

6 Proof of Concept 40
6.1 PoC Minimum Expected by the Assignment . . . 40
6.2 Delivered MVP Features Beyond the Minimum PoC . . . 40
6.3 Repository and README Evidence . . . 41
6.4 Verification Strategy . . . 41
6.5 Running the AI Flow with LM Studio . . . 41

7 Delivered MVP Alignment Notes 43
7.1 Design Assumptions vs Delivered MVP . . . 43
7.2 Why These Differences Are Defensible . . . 43

8 Conclusion 45

A Appendix 46
A.1 Submission Artifact Inventory . . . 46
A.2 Assignment Coverage Map . . . 46
A.3 Version-Control and Collaboration Evidence . . . 47
A.4 Demo Checklist . . . 48
A.5 Mermaid Diagram Source Listings . . . 48
A.5.1 C4 Level 1 — System Context (Mermaid Source) . . . 48
A.5.2 C4 Level 2 — Container Diagram (Mermaid Source) . . . 49
A.5.3 C4 Level 3 — Backend API Component Diagram (Mermaid Source) . . . 49
A.5.4 Entity-Relationship Diagram (Mermaid Source) . . . 50
A.6 Additional Mermaid Source Artifacts Available in the Repository . . . 52

Chapter 1

Executive Summary

This project designs and validates a collaborative document editor with an integrated AI writing assistant. The target product is intentionally framed as a simplified Google Docs competitor: multiple users can open the same document, see each other's edits in real time, manage sharing and access, invoke AI assistance on selected text, inspect document history, and export the result. The assignment brief asks not only for a prototype, but for evidence of disciplined requirements engineering, architecture, repository organization, and team coordination.

The final system follows a modular monorepo structure with distinct frontend, backend, realtime, AI-service, shared, and documentation layers. The frontend is a React application focused on document editing, collaboration, permissions, AI suggestions, version history, and export. The backend is an Express API responsible for authentication, document lifecycle management, role-based access control, AI orchestration, export jobs, versioning, and realtime session issuance. The realtime service is a dedicated WebSocket/Yjs relay that enforces collaboration permissions and propagates awareness and content updates. Persistence is handled through SQLite for portability and deterministic setup, while AI generation is abstracted behind a provider interface supporting both a deterministic stub provider and a local LM Studio/OpenAI-compatible endpoint.

Several architectural drivers had the strongest influence on the design. First, collaborative editing had to feel responsive and correct; this drove the decision to use Yjs-based push synchronization for document state instead of periodic polling. Second, AI needed to be useful but failure-tolerant; this led to an asynchronous job model with explicit `PENDING`/`RUNNING`/`SUCCEEDED`/`FAILED` states rather than blocking inline model calls. Third, the codebase had to be maintainable by a small student team; this motivated a monorepo, shared contract notes, explicit service boundaries, and root-level orchestration scripts for installation, running, testing, and demo reset.

The delivered MVP intentionally goes beyond the assignment's minimum proof-of-concept threshold. The original PoC requirement was a working frontend communicating with a backend over at least one meaningful API, with data contracts matching the design. The final implementation still satisfies that minimum, but also demonstrates live collaboration, autosave, permission management, version history, revert, AI rewrite/summarize/translate jobs, partial AI acceptance, export jobs, Swagger/OpenAPI docs, CI automation, and Docker Compose configuration. At the same time, this report is explicit about MVP simplifications: authentication is local bearer-token login rather than a full external identity provider, the storage engine is SQLite rather than a production-scale RDBMS deployment, and the editor focuses on collaborative text rather than a full structured rich-text schema.

The report is therefore structured to do two things at once: satisfy the assignment rubric as a requirements-and-architecture deliverable, and explain how the implemented MVP maps back to those design decisions. Each major architecture section includes a delivered-MVP note where the practical implementation intentionally differs from the longer-term target architecture. This keeps the report truthful, versionable, and aligned with the actual repository and demo.

Chapter 2

Team Responsibilities and Contributions

The assignment explicitly evaluates not just technical output, but also whether the team had clear ownership boundaries and a believable plan for cross-functional work. The project was therefore organized around service ownership with shared decision-making for cross-cutting flows such as AI, permissions, realtime collaboration, and versioning.

2.1 Ownership Model

Table 2.1: Primary ownership and contribution areas

Member	Primary Ownership	Concrete Responsibilities
Tabina	Architecture and infrastructure	Architectural drivers, C4 level 1 and level 2 diagrams, feature decomposition, repository organization, methodology, milestone planning, final repository consistency
Noel	Backend and API layer	Document management requirements, user management requirements, latency/scalability targets, C4 level 3 backend component view, API contracts, auth/authorization design, realtime feasibility, backend MVP implementation
Nurtore	AI and data model	AI requirements, quota/failure handling, privacy and security NFRs, AI integration deep design, AI model strategy ADR, AI/data-model relationships
Ming Ming	Frontend and UX	Collaboration requirements, usability NFRs, user stories, frontend PoC/MVP implementation, API usability validation, role-aware UX behavior

2.2 Cross-Cutting Collaboration

Several major features deliberately crossed ownership boundaries.

- **Realtime collaboration** required coordination between Ming Ming's frontend work, Noel's backend contracts and permission model, and the dedicated realtime service design.

- **AI assistance** touched frontend UX, backend policy enforcement, the AI provider abstraction, the data model, and versioning logic.
- **Permissions and sharing** required agreement between the frontend role UX, backend RBAC enforcement, and realtime write restrictions.
- **Final demo reliability** required infrastructure-oriented work such as shared runtime configuration, root scripts, Docker Compose, CI, and deterministic reset behavior.

To avoid fragmented decision making, the team treated backend contracts, realtime behavior, and AI interaction flows as interface boundaries that had to be reviewed jointly. This mattered because the assignment's PoC requirement was not just "make something run," but "make the code reflect the architecture document." Shared ownership therefore meant aligning contracts and assumptions, not blurring responsibility.

2.3 Decision-Making Process

Technical disagreements were resolved with a bias toward the smallest correct design that preserved clarity, demo reliability, and grading alignment. In practice, the decision rule was:

1. identify the architectural driver being optimized (correctness, latency, maintainability, cost, or demo reliability),
2. prefer the approach that reduced implementation risk while still satisfying the assignment,
3. record the choice through an ADR when the trade-off affected multiple parts of the system.

This is why the delivered MVP uses local bearer-token authentication rather than a full OAuth/OIDC integration, SQLite rather than a larger operational database deployment, and a provider abstraction with a deterministic stub AI path for demos. These choices intentionally optimize feasibility and traceability without abandoning the target architecture described in the report.

Part I

Requirements Engineering

Chapter 3

Requirements Engineering

This chapter translates the assignment brief into a structured requirements specification for the collaborative editor system. The intent is to describe not only the feature surface, but also the measurable quality constraints that shaped the final architecture and MVP implementation.

3.1 Stakeholder Analysis

The assignment explicitly asks for stakeholder categories beyond the generic "end user." The final system is influenced by actors across the entire product lifecycle, including security, deployment, operations, and the economics of AI integration.

Table 3.1: Stakeholder analysis

Stakeholder	Goals	Concerns	Design Influence
Administrators / team leads	Control access, sharing, and AI usage policies across documents and collaborators	Unauthorized edits, accidental sharing, excessive AI cost, lack of auditability	Drive RBAC, permission workflows, version history, revert, AI policy controls, and audit logging
Infrastructure / operations team	Keep services reproducible, testable, observable, and easy to deploy locally	Runtime drift, broken environments, fragile demos, unclear service boundaries	Drive Docker Compose, CI, root orchestration scripts, shared config, and simple deployable service boundaries
AI provider / model platform	Receive valid requests, maintain predictable quotas, and avoid harmful or oversized prompts	Expensive requests, privacy leakage, malformed payloads, latency spikes, and unavailable model endpoints	Drive async AI jobs, prompt scoping to selected text, quota controls, error handling, and the stub/LM Studio provider abstraction
Development team	Maintain a modular codebase that multiple people can extend safely over the semester	Tight coupling, duplicated logic, fragile contracts, and hard-to-debug collaboration bugs	Drive monorepo structure, shared contracts, repository split by domain, test coverage, and ADR documentation
Document owners / collaborators	Edit, share, review, and recover document state with minimal friction	Lost edits, confusing permissions, broken collaboration, and untrustworthy AI suggestions	Drive live collaboration, autosave, revert, partial AI apply, visibility into roles, and graceful offline/manual-save fallback

3.2 Functional Requirements

Each capability area is described first by a high-level capability statement and then by precise, testable sub-requirements with triggers, system behavior, and acceptance criteria.

3.2.1 Real-Time Collaboration

High-level capability statement. The system shall support real-time collaborative editing where multiple authenticated users can open the same document, observe each other's presence, and converge on a consistent shared document state without manual refreshes.

FR-COL-001 — Session Join and Presence Broadcast **Description.** The system shall establish a collaboration session and broadcast presence when a user opens a shared document. **Trigger.** A permitted user opens a document and the frontend requests a collaboration session. **System behavior.**

- The frontend loads the document over REST and requests a signed session from the backend.

- The realtime service validates the signed session and joins the client to the document room.
- Other collaborators receive presence/awareness updates within the collaboration channel.

Acceptance criteria.

- A successful join returns a valid session and role for the document.
- Existing collaborators observe the new participant in the presence layer within one second.
- Joining does not require a full page refresh when the user already has a valid document URL.

FR-COL-002 — Session Leave, Disconnect Detection, and Reconnect Description. The system shall degrade gracefully when a realtime connection is lost and resume collaboration when it is restored.
Trigger. The browser loses websocket connectivity, the user closes a tab, or the realtime service becomes unavailable temporarily.
System behavior.

- The frontend detects the broken realtime connection and switches into an offline/manual-save mode without losing visible local edits.
- The realtime client retries connection automatically.
- On reconnect, the client resynchronizes using the collaboration session model instead of silently overwriting newer persisted content.

Acceptance criteria.

- Connectivity loss is reflected in the UI within 2–10 seconds, depending on the failure mode.
- The user can continue editing locally while realtime is unavailable.
- When realtime returns, the client reconnects without requiring a full app restart in normal cases.

FR-COL-003 — Real-Time Text Synchronization and Convergence Description. The system shall propagate text updates among collaborators with deterministic convergence semantics.
Trigger. An editor inserts, deletes, or replaces document text during an active session.
System behavior.

- The initiating client updates local editor state immediately.
- The realtime layer relays Yjs synchronization updates to other participants.
- All connected collaborators converge to the same logical document state.

Acceptance criteria.

- Remote collaborators observe the update within the target latency budget under normal network conditions.
- Simultaneous edits do not silently discard one collaborator's changes.
- All connected clients converge to the same content after the final update is applied.

FR-COL-004 — Presence, Cursor, and Selection Awareness Description. The system shall communicate which collaborators are online and where their focus is.
Trigger. A collaborator moves the cursor, updates a text selection, or joins/leaves the room.
System behavior.

- The frontend sends awareness metadata over the realtime channel.
- Other clients render identifiable collaborator presence and selection state.

Acceptance criteria.

- Presence indicators remain distinguishable when multiple collaborators are online.
- Anonymous awareness records are not rendered as fake users.
- Presence updates do not block typing or document rendering.

FR-COL-005 — Role-Based Edit Enforcement in Collaboration Description. The system shall block unauthorized collaborative writes while allowing read-only participants to remain in the session.
Trigger. A viewer or revoked collaborator attempts to send an edit operation.
System behavior.

- The realtime service checks the current role before accepting write operations.
- Viewers continue receiving document and presence updates, but their edits are rejected.
- Role changes take effect without restarting services.

Acceptance criteria.

- Viewer write attempts do not modify shared document state.
- Authorized editor operations are accepted and propagated.
- Permission changes affect collaboration behavior promptly.

3.2.2 Document Management

High-level capability statement. The system shall manage the full document lifecycle — creation, retrieval, persistence, versioning, revert, and export — while preserving correctness under concurrent collaboration and enforcing role-based access control.

FR-DOC-001 — Create Document Description. The system shall allow an authenticated user to create a new document with initial content and ownership metadata.
Trigger. A signed-in user clicks *New Document* or submits a create request.
System behavior.

- Validate authentication and request payload.
- Create a unique `documentId`, timestamps, and an initial version entry.
- Assign the requester as the owner.

Acceptance criteria.

- Returns `201` with `documentId`, `title`, `ownerId`, `createdAt`, `updatedAt`, and `currentVersionId`.
- Invalid input returns `400 INVALID_INPUT`.
- Unauthenticated requests return `401 AUTH_REQUIRED`.

FR-DOC-002 — Retrieve Document with Permission Enforcement Description. The system shall return document content and metadata only to authorized users.
Trigger. A client requests `GET /documents/:documentId`.
System behavior.

- Authenticate the request.
- Resolve the document-scoped role.
- Return the latest persisted content, version marker, and caller role.

Acceptance criteria.

- Authorized users receive `200` with `documentId`, `title`, `content`, `updatedAt`, `currentVersionId`, `role`, and `revisionId`.
- Unauthorized access returns a consistent `403 PERMISSION_DENIED` or `404 NOT_FOUND` policy.

FR-DOC-003 — Update Document Content with Idempotent Save Description. Editors and owners shall be able to persist content using idempotent request IDs.
Trigger. The frontend performs manual save or autosave.
System behavior.

- Validate authentication, authorization, payload size, and request ID.
- If the request ID has already succeeded, return the stable previous success response.
- Otherwise, persist the content and produce a new revision marker.

Acceptance criteria.

- Viewers receive `403 PERMISSION_DENIED`.
- Repeated requests with the same request ID do not create duplicate state.
- Successful saves return `200` with `documentId`, `updatedAt`, and `revisionId`.

FR-DOC-004 — Version Snapshot Policy Description. The system shall create immutable snapshots to support auditability and rollback.
Trigger. Document creation, content persistence, revert, and AI-apply operations.
System behavior.

- Create an initial version at document creation time.
- Create version snapshots for saves and before destructive or AI-driven changes.
- Preserve monotonically increasing version numbers per document.

Acceptance criteria.

- Every document has at least one version.
- Versions are immutable once written.
- Revert and AI apply produce additional version entries rather than rewriting history.

FR-DOC-005 — List Version History Description. The system shall allow authorized users to browse version history without loading every full snapshot by default.
Trigger. A client requests `GET /documents/:documentId/versions`.
System behavior.

- Authenticate and authorize as viewer or above.
- Return version metadata sorted newest-first.
- Support pagination through limit and cursor.

Acceptance criteria.

- The response includes `versionId`, `versionNumber`, `createdAt`, `createdBy`, and `reason`.
- Default history responses exclude full version content.

FR-DOC-006 — Revert to Version Description. The system shall revert a document to a prior version while preserving an audit trail.
Trigger. The owner selects *Revert* on a version history entry.
System behavior.

- Validate authorization for revert.
- Create a pre-revert backup snapshot.
- Replace current content with the selected version's content.
- Create a new version entry representing the revert.
- Notify active realtime sessions so collaborators converge.

Acceptance criteria.

- Revert creates a new version instead of overwriting history.
- Non-existent target versions return `404 VERSION_NOT_FOUND`.
- Unauthorized revert attempts return `403 PERMISSION_DENIED`.

FR-DOC-007 — Export Document Description. The system shall support lightweight synchronous exports and heavier asynchronous export jobs.
Trigger. A user selects an export format in the UI.
System behavior.

- Validate access rights.
- Return immediate metadata for lightweight exports.
- Return an export job for heavier formats when necessary.

Acceptance criteria.

- Lightweight exports return success synchronously.
- Async exports return `202` with `jobId` and `statusUrl`.
- Download metadata includes an expiry boundary.

FR-DOC-008 — Audit Logging for Critical Document Actions Description. The system shall persist an immutable audit record for critical document actions.
Trigger. Create document, update permissions, revert, export, or apply/reject AI output.
System behavior.

- Append an audit event with actor, document, action type, timestamp, and metadata.
- Keep audit writes server-side and non-optional for the listed actions.

Acceptance criteria.

- Audit events exist for the listed security-relevant actions.
- The log is append-only from the standard application path.

3.2.3 User Management and Authorization

High-level capability statement. The system shall authenticate users and enforce document-scoped authorization consistently across REST endpoints, realtime collaboration, AI usage, and sharing flows.

FR-USER-001 — Authenticate User and Establish Session Description. The system shall authenticate a user and issue a token required for protected API access.
Trigger. A user signs in via the frontend.
System behavior.

- Validate the user identifier against the local MVP auth model.
- Return bearer-token metadata and a minimal profile.

Acceptance criteria.

- Successful auth returns `200` with `userId`, `displayName`, `accessToken`, and `expiresIn`.
- Invalid credentials or malformed auth requests fail with a structured error.

FR-USER-002 — Enforce Authorization on Every Protected Request Description. The system shall enforce server-side authorization on all protected resources.
Trigger. Any protected REST request for documents, versions, AI, permissions, export, or sessions.
System behavior.

- Validate the bearer token.
- Resolve document role and verify the requested action.
- Deny unauthorized actions without side effects.

Acceptance criteria.

- Missing or invalid auth returns `401`.
- Forbidden actions return `403 PERMISSION_DENIED`.
- Server-side checks remain authoritative even if UI controls are hidden or bypassed.

FR-USER-003 — Grant or Update Document Permissions Description. Owners shall be able to grant or update collaborator roles on a document.
Trigger. The owner submits a permission change request.
System behavior.

- Validate that the target user and requested role are acceptable.
- Create or update the document permission entry.
- Emit audit and realtime state update signals.

Acceptance criteria.

- Success returns `200` with `documentId`, `targetUserId`, `role`, and `updatedAt`.
- Non-owner attempts fail with `403 PERMISSION_DENIED`.

FR-USER-004 — Revoke Document Access Description. Owners shall be able to revoke a collaborator's access.
Trigger. The owner removes a user from the share panel.
System behavior.

- Remove the permission entry.
- Emit an audit event and notify the realtime layer.
- Deny further writes from the revoked user.

Acceptance criteria.

- Revoked users lose access through subsequent REST or realtime operations.
- Active realtime sessions are updated promptly.

FR-USER-005 — Role-Based Editing Enforcement in Realtime Sessions Description. Realtime collaboration shall enforce document roles, not just the REST layer.
Trigger. A client sends a collaborative write frame over WebSocket.
System behavior.

- Validate the signed session.
- Map the session to the current document role.
- Reject write operations from viewers.

Acceptance criteria.

- Viewer edits do not affect the shared Yjs state.
- Authorized editor writes propagate normally.

FR-USER-006 — AI Policy Controls and Quota Enforcement Description. The system shall support document-scoped AI policy controls and quota enforcement.
Trigger. An owner changes AI policy or a user invokes an AI action.
System behavior.

- Maintain document-level policy fields including AI enablement, allowed roles, and daily quota.
- Enforce policy before dispatching AI requests.
- Log AI usage for audit and cost tracking.

Acceptance criteria.

- Disabled AI returns `403 AI_DISABLED`.
- Forbidden roles return `403 AI_ROLE_FORBIDDEN`.
- Quota exhaustion returns `429 QUOTA_EXCEEDED`.

FR-USER-007 — Standard Error Response Schema Description. All non-success API responses shall use a consistent JSON error contract.
Trigger. Validation, authentication, authorization, quota, not-found, or processing failures.
System behavior.

- Return `{ error: { code, message, details? } }` consistently.
- Keep machine-readable codes stable across endpoints.

Acceptance criteria.

- All non-2xx responses follow the standard error schema.
- Shared error codes remain consistent across REST routes.

3.2.4 AI Assistant

High-level capability statement. The system shall allow users to invoke AI assistance on selected text for rewrite, summarization, and translation workflows while preserving document control, cost visibility, and collaboration safety.

FR-AI-001 — Rewrite Selected Text Description. The system shall create an asynchronous AI rewrite job for selected text.
Trigger. An editor or owner selects text and invokes rewrite.
System behavior.

- Validate permissions and quota.
- Capture the selected text plus minimal surrounding context.
- Create and run an async AI job.
- Return a suggestion without modifying the document automatically.

Acceptance criteria.

- Job creation returns quickly with `jobId` and `statusUrl`.
- Original text remains unchanged until the user explicitly applies a suggestion.

FR-AI-002 — Partial Acceptance of AI Suggestions **Description.** Users shall be able to apply all or only part of an AI suggestion.
Trigger. The user reviews a completed AI suggestion in the side panel.
System behavior.

- Present the suggestion in an editable review panel.
- Allow the user to apply the entire suggestion or only a selected portion.
- Create AI-related version snapshots before persistence.

Acceptance criteria.

- Partial acceptance replaces only the selected region in the current document range.
- Applying AI creates additional version records for traceability.

FR-AI-003 — AI Failure Handling **Description.** AI failures shall be isolated so they never corrupt document state.
Trigger. The provider times out, returns malformed output, or rejects a request.
System behavior.

- Mark the job as failed.
- Return a structured error state to the client.
- Leave the document unchanged.

Acceptance criteria.

- Failures surface as `AI_FAILED`, `AI_TIMEOUT`, or `QUOTA_EXCEEDED` rather than generic errors.
- Retry does not corrupt document state.

FR-AI-004 — Text Summarization **Description.** The system shall summarize selected text as an asynchronous AI suggestion.
Trigger. The user selects text and invokes summarize.
System behavior.

- Send selected text and scoped context to the AI provider.
- Return a summary suggestion that is not auto-applied.

Acceptance criteria.

- The summary is presented for review rather than applied directly.
- The action follows the same async job contract as rewrite.

FR-AI-005 — Text Translation **Description.** The system shall translate selected text into a requested target language.
Trigger. The user selects text, chooses a language, and invokes translate.
System behavior.

- Send the selected text and target language to the AI provider.
- Return the translated result as a reviewable suggestion.

Acceptance criteria.

- The translated output is not auto-applied.
- The same error and quota semantics apply as for rewrite/summarize.

3.3 Non-Functional Requirements

3.3.1 Latency and Scalability

- **NFR-LS-001 — Document open latency.** For documents up to 50 KB, the target p95 for `GET /documents/:documentId` is $\leq$ 300 ms, excluding client rendering.
- **NFR-LS-002 — Save latency.** For typical saves up to 10 KB, the target p95 for `PUT /documents/:documentId/content` is $\leq$ 200 ms, with idempotent replays returning even faster.
- **NFR-LS-003 — Realtime propagation.** Under normal network conditions, remote edit propagation should remain in the 200–500 ms range, with presence/cursor updates remaining visually responsive.
- **NFR-LS-004 — Version history latency.** History pages up to 200 versions should be returned within $\leq$ 300 ms p95 per page.
- **NFR-LS-005 — AI responsiveness.** AI job creation should return within $\leq$ 500 ms, while selection-based AI completion should remain within a usable 10-second target in normal local or compatible-provider conditions.
- **NFR-LS-006 — Collaboration capacity.** The target architecture should support at least 25 connected users per document and at least 10 simultaneous editors. The delivered MVP is optimized for correctness and demonstrability rather than load-tested production scale.

3.3.2 Security, Availability, and Privacy

- **NFR-SAP-001 — In-transit and at-rest protection.** The target architecture assumes secure transport and encrypted persistence in production. The delivered MVP uses localhost HTTP/WebSocket and SQLite for coursework scope and explicitly documents this simplification.
- **NFR-SAP-002 — Server-side enforcement.** All protected operations must validate bearer authentication and document-scoped authorization on the server, not only in the UI.
- **NFR-SAP-003 — AI data minimization.** By default, only the selected text plus minimal surrounding context is sent to the AI provider.
- **NFR-SAP-004 — Graceful AI degradation.** AI provider failures must not block editing, collaboration, or export workflows.
- **NFR-SAP-005 — Auditability.** Sensitive actions such as permission changes, exports, reverts, and AI feedback should create immutable server-side audit records.

3.3.3 Usability and Accessibility

- **NFR-U-001 — Collaborative clarity.** Presence indicators must remain understandable with multiple users connected.
- **NFR-U-002 — AI suggestion readability.** AI suggestions must be visually separated from the original content and easy to review or reject.
- **NFR-U-003 — Editing responsiveness.** Local typing should feel immediate and should not be blocked by background synchronization or AI processing.

- **NFR-U-004 — Large-document usability.** The UI should remain interactive for typical coursework-scale documents even when history, AI panels, and multiple collaborators are present.
- **NFR-U-005 — Error clarity.** Permission, quota, network, and conflict errors must give the user a clear explanation and a sensible next action.

3.4 User Stories and Scenarios

The system is driven by collaboration, AI, document lifecycle, and role-sensitive stories rather than a single-user editor mindset.

US-01 Offline Reconnect As an editor, I want to continue editing during a temporary loss of realtime connectivity and recover safely when collaboration resumes.

- The editor must surface an offline/manual-save mode rather than pretending collaboration is still live.
- On reconnect, the client should rejoin collaboration without silently overwriting newer persisted content.

US-02 Simultaneous Edit Conflict As an editor, I want concurrent edits to converge predictably so that other users' changes do not silently overwrite my own.

- Both collaborators' edits should survive normal concurrent editing.
- The system should converge instead of forcing one user to refresh manually after every conflict.

US-03 AI Partial Accept As an editor, I want to accept only a useful part of an AI suggestion so I can keep the good portion without applying everything.

- The user must be able to apply all of the AI output or only a selected portion.
- Applying AI should create traceable version snapshots rather than bypassing history.

US-04 AI Quota Exceeded As a user, I want a clear quota-exceeded response so I understand why an AI action did not run.

- Quota errors should be explicit rather than generic server failures.
- Editing and collaboration should continue to work even when AI is denied.

US-05 Viewer Tries to Edit As a viewer, I want the system to block editing gracefully while still allowing me to watch live updates.

- The viewer UI must remain readable and collaborative, but write attempts must fail.
- The same restriction must be enforced server-side and in realtime.

US-06 Share Document with Read-Only Access As an owner, I want to grant read-only access so others can review without editing.

- The owner must be able to grant or revoke viewer/editor access without restarting the system.
- The target user should see the new role reflected in both REST and realtime behavior.

US-07 AI Rewrite During Active Collaboration As an editor, I want to request an AI rewrite without freezing the whole document for other collaborators.

- The AI should run against a snapshot while collaborators continue editing.
- The returned suggestion should be reviewable rather than auto-applied into a moving document.

US-08 Revert to Previous Version During Collaboration As an owner, I want to revert safely and bring all collaborators to the reverted state.

- Revert must create a new version rather than destroying history.
- Active collaborators should converge to the reverted state through the realtime layer.

US-09 AI Suggestion Rejected As an editor, I want to reject an AI suggestion without changing document content.

- Rejection should be immediate and should not alter the document or undo stack.
- Rejection should still be visible in audit/feedback history.

US-10 Admin Disables AI Features As a policy owner, I want to disable AI for a document or role so the system respects cost and privacy constraints.

- AI-disallowed states should be enforced before any provider call is made.
- The UI should explain that AI is disabled instead of silently hiding failure.

US-11 Create Document As an authenticated user, I want to create a new document and become its owner immediately.

- Creation should also initialize a first version snapshot and ownership record.
- The frontend should redirect directly to the created document.

US-12 Export Document As a collaborator with access, I want to export a document in a supported format for external sharing or review.

- Lightweight formats may complete immediately; heavy formats should use an async job.
- Export permissions must follow the same document role model as the rest of the system.

US-13 AI Summarize and Translate As an editor, I want to summarize or translate selected text as reviewable suggestions.

- The AI output should be shown for review rather than inserted automatically.
- Both summarize and translate should follow the same async job and permission model as rewrite.

3.5 Requirements Traceability

The traceability matrix in Table 3.2 links the user stories above to the functional requirements and then to the architectural components that implement them. This matrix was maintained as an explicit anti-drift artifact because the MVP evolved significantly beyond the initial PoC baseline.

Table 3.2: Requirements traceability matrix

User Story	**Functional Requirement ID(s)**	**Architecture Component(s)**
US-01 Offline reconnect	FR-COL-002, FR-COL-003, FR-DOC-003, FR-USER-005	Frontend, Realtime Service, Backend API, Database
US-02 Simultaneous edit conflict	FR-COL-003, FR-DOC-003, FR-DOC-004	Frontend, Realtime Service, Backend API, Database
US-03 AI partial accept	FR-AI-001, FR-AI-002, FR-DOC-004	Frontend, AI Integration Service, Backend API, Database
US-04 AI quota exceeded	FR-AI-003, FR-USER-006	Backend API, AI Integration Service
US-05 Viewer tries to edit	FR-USER-001, FR-USER-002, FR-USER-005, FR-USER-007, FR-DOC-002	Frontend, Backend API, Realtime Service
US-06 Share document with read-only access	FR-USER-003, FR-DOC-002, FR-USER-004	Frontend, Backend API, Database
US-07 AI rewrite during active collaboration	FR-AI-001, FR-AI-003, FR-COL-003	Frontend, AI Integration Service, Realtime Service, Backend API
US-08 Revert to previous version during collaboration	FR-DOC-006, FR-DOC-005, FR-DOC-004	Backend API, Realtime Service, Database
US-09 AI suggestion rejected	FR-AI-002, FR-AI-003	Frontend, AI Integration Service, Backend API
US-10 Admin disables AI features	FR-USER-006	Backend API, AI Integration Service
US-11 Create document	FR-DOC-001	Frontend, Backend API, Database
US-12 Export document	FR-DOC-007, FR-DOC-008	Backend API, Database

User Story	Functional Requirement ID(s)	Architecture Component(s)
US-13 AI summarize and translate	FR-AI-004, FR-AI-005	Frontend, AI Integration Service, Backend API

Part II

System Architecture

Chapter 4

System Architecture

4.1 Architectural Drivers

The architecture was not chosen in a vacuum. The most important design drivers were ranked before the service boundaries were fixed because different priorities would have produced a meaningfully different system.

Table 4.1: Ranked architectural drivers

Rank	Driver	Why it mattered
1	Collaboration correctness and convergence	The product fails if two users cannot trust what they are seeing while editing the same document. This pushed the design toward a CRDT-backed push model rather than naive polling or last-write-wins merging.
2	Low-latency user experience	Typing, presence, and AI interaction feedback must feel responsive enough to resemble a real collaborative editor, not a batch workflow.
3	Maintainable modular codebase	The assignment explicitly evaluates repository organization and feature decomposition. The architecture had to support multiple owners working in parallel without turning into a god-service monolith.
4	AI safety, predictability, and cost control	AI is a core product feature, not a novelty. This drove the asynchronous job model, scoped prompting, and explicit policy/quota controls.
5	Demo and reproducibility reliability	The MVP had to be runnable under grading pressure. This drove root orchestration scripts, SQLite portability, Docker Compose, CI, and deterministic reset behavior.

These rankings directly shaped the implementation: the realtime service is separated from the REST API, AI uses asynchronous job handling rather than inline blocking calls, persistence is centralized in SQLite for deterministic setup, and the repository is organized by service and domain boundaries rather than by one large application file.

4.2 C4 Level 1 — System Context

Figure 4.1 shows the system at the outermost level. The collaborative editor interacts with human collaborators, a target identity provider, and an LLM provider. This level answers

the question: what is inside the system boundary, and what external actors or services affect behavior?

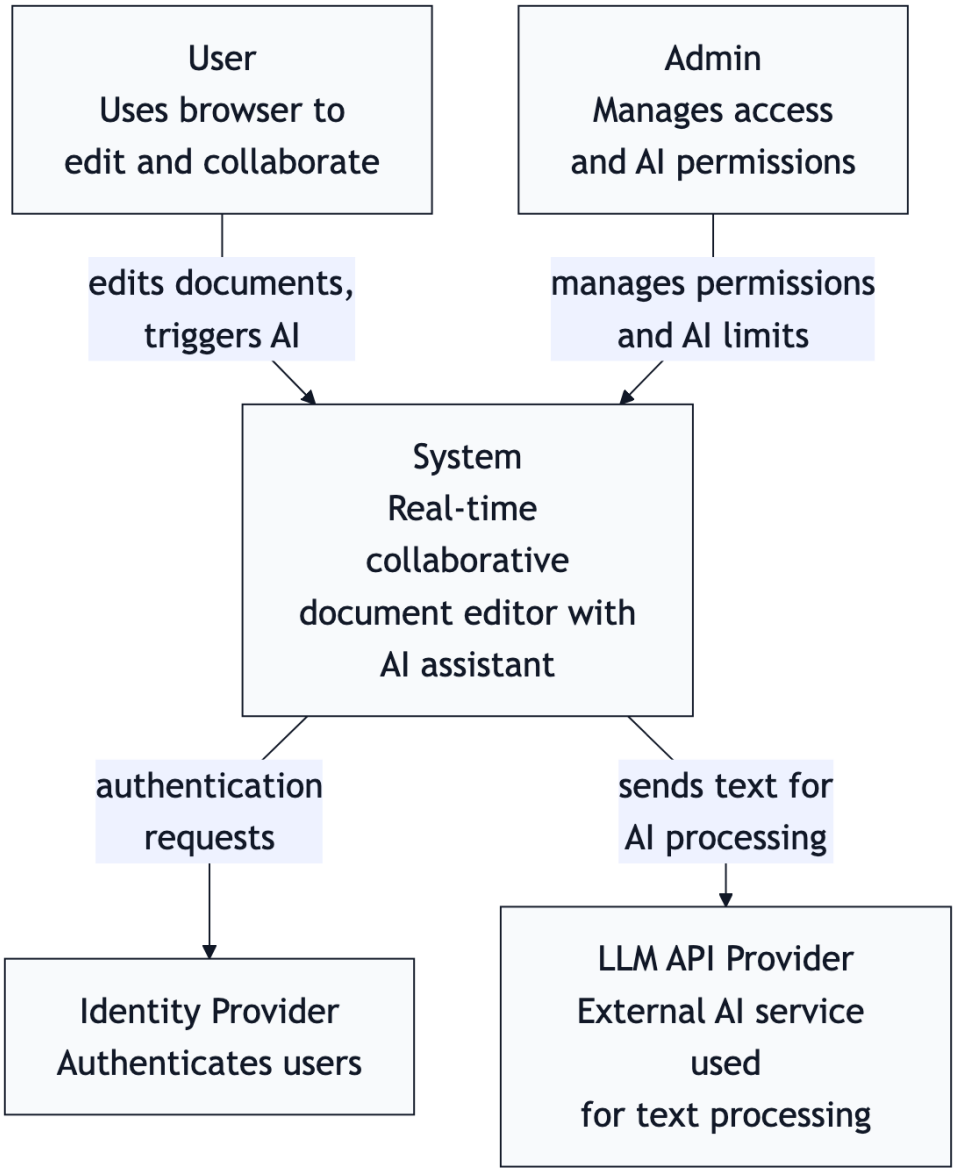

Figure 4.1: C4 Level 1 — System context diagram

Delivered MVP implementation note. The delivered MVP uses a visible local login flow backed by `POST /auth/login` rather than a full external OAuth/OIDC identity provider. The system-context diagram therefore captures the target production-style identity boundary, while the MVP demonstrates the same trust boundary with a simpler bearer-token implementation suitable for coursework and demos.

4.3 C4 Level 2 — Container Diagram

Figure 4.2 decomposes the system into the major runtime containers. The frontend is the user entry point. The backend API owns document lifecycle, permissions, versioning, export, AI orchestration, and session issuance. The realtime service is dedicated to Yjs synchronization, awareness, and role-aware write gating. The AI integration service abstracts provider selection,

and the database stores the shared system state.

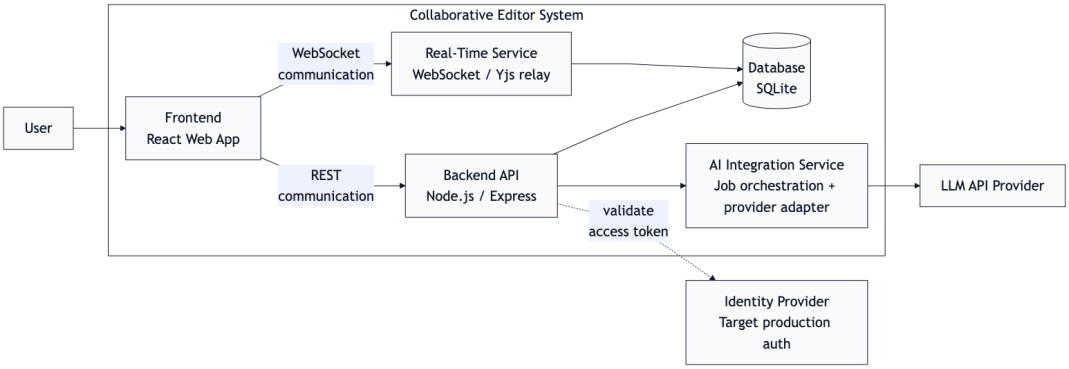

Figure 4.2: C4 Level 2 — Container diagram

Delivered MVP implementation note. The delivered MVP uses SQLite rather than a production-scale database deployment. The architecture keeps the same container responsibilities, but the concrete persistence technology is optimized for a reproducible academic environment. The realtime service and backend share the same SQLite state to keep sessions, permissions, versions, and backend-driven collaboration events aligned.

4.4 C4 Level 3 — Backend API Component Diagram

The backend was chosen as the container to zoom into because it contains most of the assignment's cross-cutting concerns: document CRUD, RBAC, AI job orchestration, export job orchestration, versioning, and realtime session issuance. Figure 4.3 shows the controller/service/repository split used to keep the API layer maintainable.

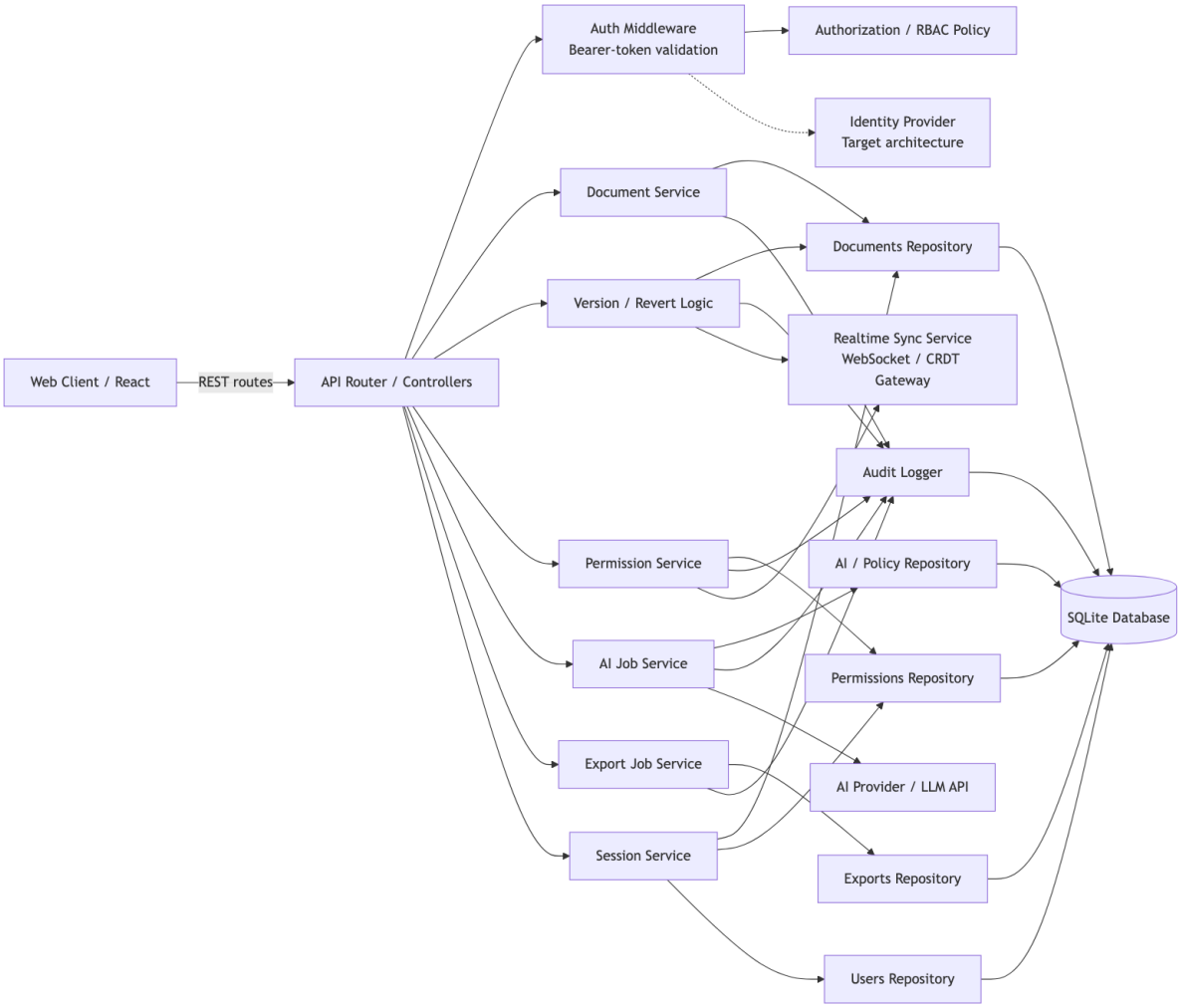

Figure 4.3: C4 Level 3 — Backend API component diagram

Delivered MVP implementation note. The delivered backend closely follows this structure: routes/controllers handle HTTP translation, auth middleware validates bearer tokens, services encode document/AI/export/session rules, repositories isolate SQL, and the realtime service is notified through document-scoped events rather than through direct in-process mutation. The MVP uses SQLite and local bearer-token authentication in place of the fuller target stack shown in the conceptual diagram.

4.5 Feature Decomposition

The architecture is organized around features/modules rather than a single monolithic application file. Each module exposes a narrow interface, depends on a small set of collaborators, and can be tested in relative isolation.

Table 4.2: Feature decomposition

Module	**Primary Responsibility**	**Dependencies / Exposed Interface**
Frontend editor and state	Render document content, local editing state, AI review panel, permissions, history, export controls	Depends on Backend API and Realtime Service; exposes user actions as REST and websocket calls
Realtime synchronization layer	Relay Yjs sync and awareness frames, enforce role-aware writes, react to backend permission/revert events	Depends on signed sessions, shared DB state, and frontend websocket clients; exposes session-ready and live sync behavior
Backend API layer	Authenticate users, enforce RBAC, manage document lifecycle, issue realtime sessions, orchestrate AI and exports	Depends on repositories, AI provider adapter, and realtime-facing event model; exposes REST contracts and standard error schema
AI integration service	Translate user AI requests into provider calls, manage async jobs, unify error handling	Depends on provider implementation and backend persistence; exposes job creation and polling semantics
Document storage and versioning	Persist documents, versions, policies, jobs, permissions, idempotency keys, and audit records	Depends on SQLite; exposes repository methods to backend services
Shared contract/utilities layer	Session-token helpers and reusable cross-service logic	Depends on no single service; exposes shared helpers used by backend and realtime

4.6 AI Integration Design

The AI assistant is treated as a core product feature, not a cosmetic API add-on. That means the design must answer four questions simultaneously: what context the model sees, how users review and apply results, how collaboration behaves while the AI is running, and how cost and failure are controlled.

4.6.1 Context and Scope

The delivered MVP sends the selected text plus a small amount of surrounding context to the AI provider. This is the most pragmatic choice for coursework: it is cheaper than sending the full document, keeps latency lower, and aligns with how rewrite/summarize/translate are actually invoked in the UI. The backend contract requires the frontend to send a client-side selection snapshot containing the selected range, the selected text, and the nearby context so that AI jobs can be linked to a concrete document state.

For very long documents, the system intentionally does not default to sending the entire document. This keeps token usage bounded and reduces privacy exposure. A future production design could add chunked/restructure flows, but the delivered MVP focuses on scoped selection-based tasks that are easier to reason about and test.

4.6.2 Suggestion UX

AI suggestions are presented in a side panel rather than silently modifying the document. This preserves user control and matches the assignment's emphasis on AI as a product feature rather than a hidden automation step. The delivered MVP supports:

- full apply of the AI output,
- partial apply by selecting only a portion of the AI output,
- explicit rejection without modifying document content,
- immediate visibility of AI job state.

Applying AI is treated as a document-changing action and therefore creates version snapshots for traceability. This design also aligns with the version history and revert model: AI is not special-cased into an untracked side path.

4.6.3 AI During Collaboration

The design intentionally avoids hard-locking a region when AI is invoked. Instead, the system snapshots the selected text, lets the AI run asynchronously, and allows other collaborators to continue editing. When the AI response arrives, the frontend presents it as a proposal. If the relevant text has changed in the meantime, the user must explicitly decide whether to apply or discard the AI output. This is a reconcile-first design rather than a lock-first design.

This choice is justified by the architectural drivers. Locking would be simpler to explain, but it would damage collaboration fluidity and create awkward waiting states. Reconciliation with explicit user review keeps the collaboration model more honest, even if it requires careful versioning and conflict-safe save behavior.

4.6.4 Prompt Design and Provider Strategy

Prompts are task-specific and template-based. The frontend chooses the user-facing action (rewrite/summarize/translate), while the backend/provider layer is responsible for constructing the final provider request. This keeps prompt construction out of the browser and allows prompt logic or provider selection to evolve without changing the frontend contract.

The delivered MVP uses a provider abstraction with two concrete modes:

- `stub`: a deterministic provider used for reliable demos and tests,
- `lmstudio`: a local OpenAI-compatible provider used when a real local model endpoint is available.

This is a pragmatic compromise between the architectural goal of AI usefulness and the assignment's need for repeatable demonstrations.

4.6.5 Async Job Model

AI operations are modeled as asynchronous jobs. The backend validates access, policy, quota, and input shape, creates a job record, and returns quickly with `jobId`, `statusUrl`, and an initial status. The frontend polls until the job reaches `SUCCEEDED` or `FAILED`. This model avoids blocking request threads and gives the client a precise way to distinguish "still running" from "failed" and "succeeded."

4.7 API Design

The API style follows a hybrid model:

- **REST** for CRUD, permissions, versions, exports, AI jobs, and session issuance,
- **WebSocket + Yjs** for low-latency collaborative state propagation,
- **Polling** from the frontend only for long-running AI jobs and export jobs.

This division is deliberate. REST keeps persistence-oriented operations predictable and easy to test. Realtime collaboration is handled by a dedicated push channel because document editing is inherently a shared live interaction. Long-running AI and export operations fit naturally into job-based REST flows.

4.7.1 Core API Contracts

The assignment brief requires at least document CRUD, realtime session management, AI invocation, and user/permission management. The delivered backend provides these contracts concretely and documents them through OpenAPI and contract notes in the repository.

Table 4.3: Core REST contract families used by the MVP

API family	Concrete delivered endpoints
Authentication	`POST /auth/login`, `GET /health`
Document lifecycle	`POST /documents`, `GET /documents/:documentId`, `PUT /documents/:documentId/content`
Versioning	`GET /documents/:documentId/versions`, `POST /documents/:documentId/revert`
Permissions and AI policy	`GET/PUT/DELETE /documents/:documentId/permissions`, `GET/PUT /documents/:documentId/ai-policy`
Realtime session management	`POST /sessions`
AI jobs	`POST /ai/rewrite`, `POST /ai/summarize`, `POST /ai/translate`, `GET /ai/jobs/:jobId`
Export jobs	`POST /documents/:documentId/export`, `GET /exports/:jobId`, `GET /exports/:jobId/download`

All non-2xx responses use a shared JSON error shape:

```
{ "error": { "code": "<CODE>", "message": "<MESSAGE>", "details": {
...optional... } } }
```

This is important because the assignment brief explicitly asks how the client can distinguish between AI latency, AI failure, quota issues, permission problems, and not-found conditions. The standard error schema gives the frontend a uniform contract for all of those cases.

Representative delivered contracts are shown below.

Document create

```
POST /documents

Request JSON:
{ "title": "Project brief", "content": "Initial content" }
```

```
Response JSON (201):
{
  "documentId": "doc_123",
  "title": "Project brief",
  "ownerId": "user_1",
  "createdAt": "ISO-8601",
  "updatedAt": "ISO-8601",
  "currentVersionId": "ver_1"
}
```

Document load

```
GET /documents/:documentId

Response JSON (200):
{
  "documentId": "doc_123",
  "title": "Project brief",
  "content": "Latest text",
  "updatedAt": "ISO-8601",
  "currentVersionId": "ver_3",
  "role": "editor",
  "revisionId": "rev_3"
}
```

Realtime session issuance

```
POST /sessions

Request JSON:
{ "documentId": "doc_123" }

Response JSON (200):
{
  "sessionId": "sess_123",
  "wsUrl": "ws://localhost:3001/ws?token=...",
  "role": "viewer|editor|owner"
}
```

AI rewrite invocation

```
POST /ai/rewrite

Request JSON:
{
  "documentId": "doc_123",
  "selection": { "start": 0, "end": 25 },
  "selectedText": "Original selected text",
  "contextBefore": "",
  "contextAfter": " remaining content",
  "instruction": "Make this more concise",
  "baseVersionId": "ver_3",
  "requestId": "req_ai_001"
}

Response JSON (202):
{
  "jobId": "aijob_123",
  "statusUrl": "/ai/jobs/aijob_123",
```

```
  "status": "PENDING"
}
```

AI job polling

```
GET /ai/jobs/:jobId

Response JSON (200):
{
  "jobId": "aijob_123",
  "statusUrl": "/ai/jobs/aijob_123",
  "status": "SUCCEEDED",
  "proposedText": "Rewritten text",
  "baseVersionId": "ver_3",
  "createdAt": "ISO-8601",
  "updatedAt": "ISO-8601"
}
```

Permission grant

```
PUT /documents/:documentId/permissions

Request JSON:
{ "requestId": "req_perm_001", "targetUserId": "user_2", "role": "editor" }

Response JSON (200):
{ "documentId": "doc_123", "targetUserId": "user_2", "role": "editor", "updatedAt": "
    ISO-8601" }
```

4.7.2 Authentication and Authorization

The target architecture includes an external identity provider, but the delivered MVP uses an explicit local login flow backed by bearer tokens. The backend nevertheless enforces the same high-level concerns that would remain in a production system: authentication gates protected resources, authorization is document-scoped, and the UI is not treated as the source of truth.

Table 4.4: Document-scoped role matrix in the delivered MVP

Action	Viewer	Editor	Owner
View/open document	Yes	Yes	Yes
Receive realtime updates/presence	Yes	Yes	Yes
Send collaborative edit operations	No	Yes	Yes
Persist content via REST save/autosave	No	Yes	Yes
View version history	Yes	Yes	Yes
Revert to previous version	No	No	Yes
Export document	Yes	Yes	Yes
Grant/update collaborator roles	No	No	Yes
Revoke access	No	No	Yes
Invoke AI rewrite/summarize/translate	No	Yes	Yes
Configure AI policy and quota	No	No	Yes

Delivered MVP implementation note. The delivered system fully demonstrates owner/editor/viewer behavior. “Admin” is kept as a design-level role in the report because it remains meaningful for future policy and organization-level control, but the MVP’s primary demonstrated management path is document ownership.

4.7.3 Communication Model

The communication model is intentionally hybrid.

- **Document open flow.** The frontend first loads a document snapshot through REST and then requests a signed realtime session URL from the backend.
- **Live collaboration.** The realtime service validates the signed session token and relays Yjs sync and awareness updates among collaborators.
- **Disconnect behavior.** If realtime becomes unavailable, the frontend falls back to offline/manual-save mode.
- **AI completion.** AI uses async jobs with polling instead of websocket completion events in the MVP because the job semantics are easier to reason about and test.

Additional sequence-diagram artifacts for document-open/realtime-join and AI-job execution are included in the repository's editable Mermaid sources, but omitted from the main PDF to keep the report focused on the assignment's required C4 and ER views.

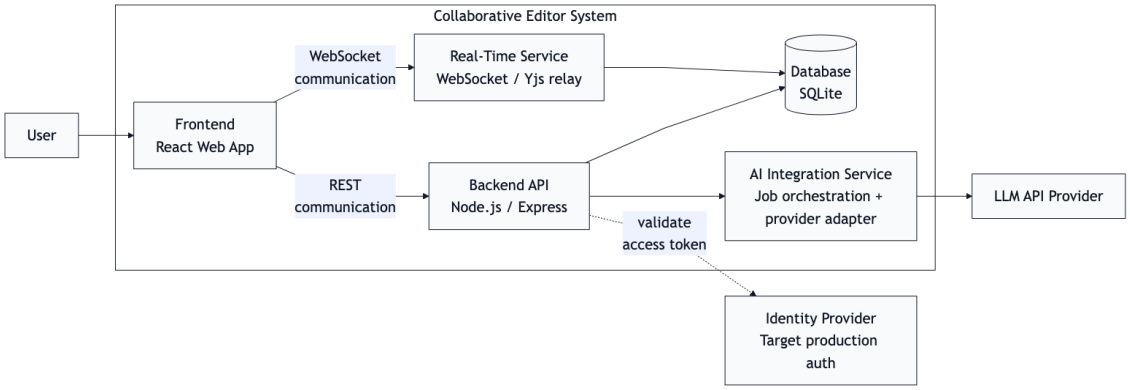

Figure 4.4: Runtime containers involved in document open and collaboration sync

Delivered MVP implementation note. The delivered system uses Yjs over WebSocket for text synchronization and presence. Polling is used only for async AI jobs and export jobs, and internally for backend-driven permission/ revert event delivery to realtime rooms. It is not used to poll text diffs every second.

4.8 Code Structure and Repository Organization

The final repository intentionally uses a monorepo. For a four-person academic team with strongly connected frontend, backend, realtime, and AI layers, a monorepo reduces coordination overhead and makes contract drift easier to detect.

4.8.1 Top-Level Layout

```
collaborative-editor-ai/
+-- frontend/        React + TypeScript client
+-- backend/         Express API, SQLite persistence, RBAC, AI jobs, export
+-- realtime/        WebSocket + Yjs relay service
+-- ai-service/      AI provider abstraction and prompt execution
+-- shared/          shared contracts such as session token signing
+-- docs/            architecture notes, Mermaid sources, diagram exports
```

```
+-- scripts/         root install/dev/test/demo orchestration
+-- tests/           root integration smoke tests
`-- final-report/    final written submission package
```

4.8.2 Shared Code and Config

The shared layer is intentionally narrow. It contains only cross-service helpers that justify shared ownership, such as session-token signing/verification. This avoids over-abstracting everything into a shared package while still keeping critical contracts consistent. Configuration is managed through environment variables and `.env.example` files; secrets are excluded from version control.

4.8.3 Testing Structure

The repository uses layered testing rather than relying on only one type of test.

- Backend: unit/integration tests with Node's built-in test runner
- Frontend: component/integration-style tests with Vitest
- Realtime: websocket/Yjs integration tests with Node's built-in test runner
- Root: a smoke integration test that starts backend and realtime together and validates wiring

This structure mirrors the feature decomposition: REST, realtime, and UI are each tested in their own layer, and the root smoke test catches cross-service drift.

4.9 Data Model

Figure 4.5 shows the core entity relationships. The design centers on documents, users, permissions, versions, and AI interactions, but the delivered MVP also persists AI policy, export jobs, realtime events, audit logs, and idempotency keys to make the runtime predictable and observable.

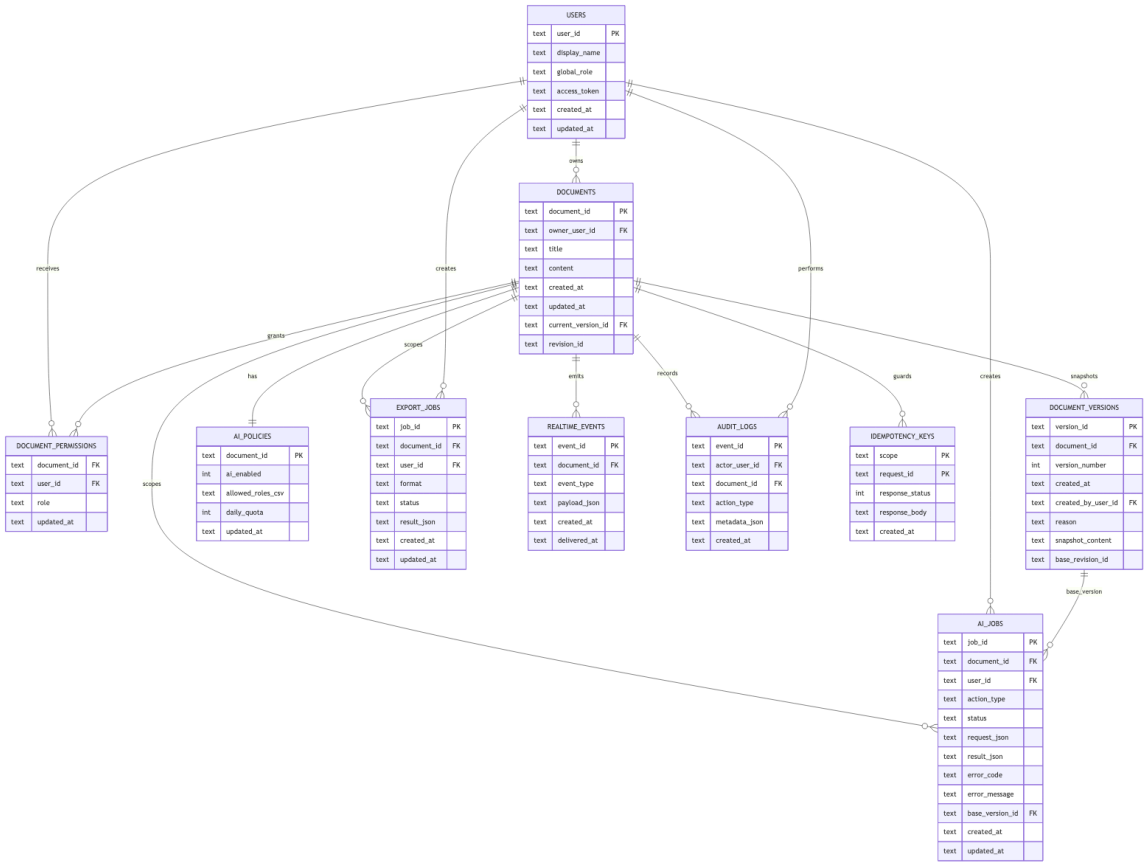

Figure 4.5: Entity-relationship diagram for the collaborative editor domain

Delivered MVP implementation note. The delivered schema uses SQLite and includes implementation-supporting entities such as AI policy, export jobs, realtime events, audit logs, and idempotency keys in addition to the core design entities. This keeps the delivered MVP honest about the data required to support jobs, permissions, versioning, and collaboration behavior.

4.10 Architecture Decision Records

ADR-001 — Realtime consistency strategy

Status: Accepted.
Context: The system needs simultaneous editing, reconnect safety, and convergence without the team implementing a custom operational transform algorithm.
Decision: Use a CRDT-based collaboration approach built on Yjs, with the realtime service acting as a relay and permission gate rather than a custom transform engine.
Consequences: This improves convergence and offline/reconnect resilience, but requires careful handling of room lifecycle, awareness state, and persistence boundaries.
Alternatives considered: Custom OT server logic, naive last-write-wins, and polling-based text sync were rejected due to complexity or poor correctness.

ADR-002 — API style and async job model

Status: Accepted.
Context: The system mixes CRUD-like document operations with long-running AI and export actions.

Decision: Use REST for document, permission, version, AI job, export job, and session endpoints; use WebSocket for realtime collaboration; and model AI/export as asynchronous jobs.
Consequences: The API is concrete and testable, but the frontend must poll job status and the backend must persist job state.
Alternatives considered: synchronous AI calls, GraphQL for all interactions, and polling for live collaboration were rejected.

ADR-003 — Repository structure strategy

Status: Accepted.
Context: The team needed a code structure that matched a multi-service system while minimizing coordination overhead for a small academic group.
Decision: Use a monorepo with service directories and keep persistence logic grouped by domain repositories rather than a single giant application repository file.
Consequences: Shared contracts are easier to keep aligned, but discipline is required to prevent tight coupling.
Alternatives considered: separate repositories and hybrid split-repo models were rejected due to contract drift and coordination cost.

ADR-004 — AI provider strategy

Status: Accepted.
Context: The system needs rewrite, summarize, and translate behaviors that are useful during a demo but not dependent on an always-available external cloud provider.
Decision: Keep AI behind a provider abstraction supporting a deterministic stub mode and a local LM Studio/OpenAI-compatible mode.
Consequences: Demo reliability improves and tests remain deterministic, but the system is still architected so a different provider can be added later.
Alternatives considered: hard-wiring one hosted provider or requiring self-hosted production-scale model infrastructure for the MVP.

Part III

Project Management and Team Collaboration

Chapter 5

Project Management and Team Collaboration

This section explains how the team intended to turn architecture into working software. The assignment brief is explicit that repository organization, ownership, code review, and risk management are part of the assessed design, not administrative afterthoughts.

5.1 Team Structure and Ownership

The team structure follows service and concern boundaries rather than assigning work randomly.

- **Tabina** owned architecture, infrastructure, repository organization, timeline, and final consistency checks.
- **Noel** owned the backend, API contracts, auth/authorization design, and realtime feasibility.
- **Nurtore** owned AI behavior, cost/privacy considerations, and the data model.
- **Ming Ming** owned frontend UX, collaboration flows, and API usability from the client side.

Features spanning multiple owners were coordinated through contract-first discussions. For example, realtime collaboration required alignment between frontend state behavior, backend role rules, and the separate realtime service; AI assistance required alignment between UI review behavior, backend async job contracts, AI provider behavior, versioning, and auditability.

When team members disagreed on a technical decision, the group resolved the disagreement by asking which architectural driver mattered most for the feature in question. For example, demo reliability and maintainability justified a deterministic stub AI provider and SQLite in the delivered MVP, even though the longer-term architecture could target a fuller identity provider and larger-scale database deployment.

5.2 Development Workflow

5.2.1 Branching Strategy

The intended workflow is feature-branch based with protected `main`. Small, purpose-specific branches reduce merge risk and keep code review focused. A representative naming convention is:

- `backend/<name>-<feature>`

- `frontend/<name>-<feature>`
- `realtime/<name>-<feature>`
- `mvp/<milestone>` for final integration work

The merge policy is PR-first: no direct pushes to `main`. Every feature branch is expected to leave the codebase runnable and testable.

Representative PR approval criteria were:

- the change compiles or runs locally,
- the touched service tests pass or a manual verification note is included,
- API contracts and docs stay aligned,
- the commit history remains small, purposeful, and reviewable.

5.2.2 Code Review Process

The team used small commits and service-scoped reviews as the default. Review criteria were:

1. does the change preserve the documented contract,
2. is server-side authorization still enforced,
3. does the feature have enough verification (tests or a justified manual check),
4. does the change reduce or increase coupling,
5. can a teammate understand the code quickly.

Backend and realtime changes were reviewed especially carefully when they affected collaboration, permissions, or contracts, because those areas tend to create cascading failures across the stack.

5.2.3 Task Tracking and Communication

Work was broken down by feature slices that mapped to ownership boundaries but were still traceable to requirements. Typical slices included: API contract establishment, document persistence, realtime session join, AI job orchestration, permission UI, and demo/deployment hardening.

Task tracking is best represented through GitHub issues or project-board items labeled by ownership area, for example `backend`, `frontend`, `realtime`, `ai`, and `docs`. This makes dependencies visible and helps the team see when one milestone is blocked by infrastructure, contracts, or UI work owned elsewhere.

In practice, the team used GitHub for branch work, pull requests, issues, and durable code review discussion, while Google Docs was used for collaborative drafting of the architecture and report content. Lightweight team chat was used for short coordination loops, but technical decisions were expected to be captured back into persistent artifacts such as ADRs, diagrams, PR discussions, README updates, and contract notes instead of being lost in chat history.

5.3 Development Methodology

The most suitable methodology for this project is a lightweight Kanban/Scrum hybrid.

- **Why not pure Scrum?** A student team with cross-cutting architecture tasks often has uneven availability and many infrastructure tasks that do not map cleanly to short user-story sprints.
- **Why not pure ad hoc Kanban?** The assignment has staged deliverables — requirements, architecture, PoC, final MVP hardening — that benefit from milestone-based planning.

In practice, the team can work in short milestone-oriented iterations while still prioritizing a Kanban board by urgency and dependency. Backlog order should favor infrastructure and contract work that unblocks multiple team members before polishing user-facing features.

Non-feature work such as writing tests, refining diagrams, splitting large files, or hardening local startup scripts is treated as first-class work rather than invisible overhead. This is especially important because the assignment explicitly evaluates repository organization, consistency, and the extent to which the PoC matches the documented architecture.

5.4 Risk Assessment

Table 5.1: Technical risk assessment

Risk	Likelihood	Impact	Mitigation	Contingency
Realtime sync divergence or duplicate updates	Medium	Conflicting document views, lost trust in collaboration demo	Use Yjs instead of a custom algorithm, centralize realtime tests, preserve room state across reconnects, regression test save/reconnect paths	Fall back to manual save mode and present conflict-safe reload if realtime becomes unstable
AI provider latency, outage, or malformed responses	High	AI actions stall or fail during demo	Async job model, provider abstraction, deterministic stub fallback, timeout/error mapping tests	Switch demo provider to stub and keep editing workflow available
Backend and realtime configuration drift	Medium	Auth/session failures and invisible data mismatch across services	Shared root scripts, shared realtime secret, shared SQLite path, Docker Compose configuration	Reset with `npm run demo:reset` and verify health/docs endpoints before demo
Permission enforcement gaps between REST and realtime	Medium	Unauthorized edits or inconsistent collaboration behavior	Server-side RBAC on every REST route and websocket write path, viewer rejection tests, permission update events	Downgrade affected users to viewer mode and block live editing until policy is revalidated
Document-report divergence late in the semester	High	Lost marks despite working code	Keep architecture notes, OpenAPI, Mermaid sources, and final report in version control; explicitly document MVP deviations	Add a final alignment appendix and demo checklist so reviewers can map code to report claims quickly

5.5 Timeline and Milestones

The project timeline must be described in terms of verifiable outcomes, not vague progress statements. The milestones in Table 5.2 reflect how the system could evolve through the semester without losing alignment between requirements, architecture, and code.

Table 5.2: Semester milestone plan

Milestone	Goal	Acceptance Criteria
M1	Requirements and architecture baseline	Stakeholders, functional requirements, NFRs, user stories, C4 L1/L2/L3, data model, and ADR drafts reviewed by the team
M2	Backend and API contract foundation	Auth, document create/load/save, permissions, sessions, AI job creation, and OpenAPI/contract notes verified by automated backend tests
M3	Realtime collaboration slice	Two users can join the same document, see presence, and converge on edits through the realtime service with viewer write protection
M4	AI and versioning integration	Async AI actions, AI policy controls, version history, revert, and export jobs demonstrated through frontend and backend integration tests
M5	Submission hardening	Root scripts, Docker Compose, CI, final report, diagram sources, README, and video script completed and reviewed against the rubric

Part IV

Proof of Concept and Delivered MVP

Chapter 6

Proof of Concept

The assignment's proof-of-concept requirement is intentionally modest: demonstrate working frontend to backend communication, validate the core API data contract, and organize the repository according to the architecture. The delivered system satisfies that minimum while going beyond it.

6.1 PoC Minimum Expected by the Assignment

The brief requires three core things:

1. a working frontend with a visible editing page,
2. at least one meaningful API call between frontend and backend,
3. request/response data contracts that match the architecture document.

The delivered repository satisfies those requirements directly:

- the frontend provides a working document editor interface,
- the backend exposes concrete routes for document creation, loading, saving, permissions, AI, sessions, versioning, revert, and export,
- the documented contracts are represented by OpenAPI, contract notes, and automated tests.

6.2 Delivered MVP Features Beyond the Minimum PoC

The final implementation goes well beyond the minimum PoC threshold while still preserving the same architectural skeleton. The delivered MVP includes:

- a visible local login experience backed by `POST /auth/login`,
- local bearer-token login through `POST /auth/login`,
- live collaboration with Yjs over WebSocket,
- autosave with idempotent backend persistence,
- version history and revert,
- document-scoped permissions and AI policy controls,
- async AI jobs for rewrite, summarize, and translate,

- partial AI acceptance,
- synchronous and asynchronous export flows,
- Swagger/OpenAPI docs at `/docs`,
- root commands for install, run, test, and deterministic demo reset,
- Docker Compose and CI support.

6.3 Repository and README Evidence

The assignment rubric explicitly checks whether the repository is organized according to the documented structure and whether a third party can run the system from the README. The delivered repository provides that through:

- a monorepo matching the documented service decomposition,
- root orchestration scripts: `install:all`, `dev:all`, `test:all`, and `demo:reset`,
- service-specific READMEs and a root README,
- contract notes and OpenAPI for backend routes,
- a deterministic smoke test that validates backend + realtime wiring.

6.4 Verification Strategy

The delivered MVP demonstrates feasibility through both automated and manual validation.

- Backend tests validate routing, auth, permissions, AI jobs, export, versions, and sessions.
- Frontend tests validate API clients, realtime behavior, AI panel interactions, and document-page workflows such as autosave and stale-save handling.
- Realtime tests validate websocket session behavior, collaborative convergence, viewer rejection, and revert propagation.
- A root integration smoke test validates that backend and realtime services work together under a shared SQLite state.

In addition, the live demo flow shows the user-facing proof that the architecture holds together: login, create a document, grant access to a second user, live collaboration, autosave, AI invocation, partial AI acceptance, version history, revert, export, and Swagger docs.

6.5 Running the AI Flow with LM Studio

The delivered MVP supports two AI-provider modes:

- `stub`: deterministic demo-safe behavior with no external dependency,
- `lmstudio`: a local OpenAI-compatible provider for a real model-backed experience.

For the strongest demo, LM Studio should be started before the application stack so the backend can use the real provider path.

Recommended LM Studio setup for the delivered MVP:

1. Open LM Studio.
2. Load a local model that supports the OpenAI-compatible chat-completions API.
3. Start the local server and keep it reachable at `http://127.0.0.1:1234`.
4. Verify that the model server responds at `/v1/models`.
5. Start the application with either:
   - `npm run demo:reset` (auto-detects LM Studio when available), or
   - `AI_PROVIDER=lmstudio npm run demo:reset` to force the LM Studio path.

This design is intentional. The stub provider guarantees reliable demos and deterministic tests, while LM Studio demonstrates the same asynchronous AI workflow against a real local model endpoint without requiring a paid hosted provider.

Practical verification commands are:

```
curl http://127.0.0.1:1234/v1/models
AI_PROVIDER=lmstudio npm run demo:reset
```

If LM Studio is unavailable, the rest of the system — editing, collaboration, versioning, permissions, and export — still remains usable. Only the AI-generation path degrades, which is exactly the graceful failure behavior described in the architecture.

Chapter 7

Delivered MVP Alignment Notes

One of the easiest ways to lose marks on this assignment is to submit a report and a codebase that look like they were built independently. The final implementation therefore needs to be explained in relation to the original design assumptions rather than pretending the first draft was already identical to the final MVP.

7.1 Design Assumptions vs Delivered MVP

Table 7.1: Key alignment notes between target architecture and delivered MVP

Design Assumption	**Delivered MVP**	**Reason for Difference**
External identity provider (OAuth/OIDC style)	Local bearer-token login via `POST /auth/login`	Keeps the proof of concept feasible while preserving the same trust boundary and role model
General relational database target	SQLite shared by backend and realtime	Portable, deterministic local setup; easier demo reset, lower infrastructure risk
Potentially hosted LLM provider	Configurable stub or local LM Studio provider	Improves reliability, cost control, and test determinism without changing the client-facing contract
PoC limited to basic frontend-backend communication	Delivered MVP includes live sync, autosave, permissions, AI, versions, revert, export, Docker, and CI	Team continued implementation beyond the minimum requirement once the architecture stabilized
Abstract collaboration transport	Concrete Yjs + WebSocket realtime service	Needed to validate simultaneous editing feasibility and collaboration correctness early
Generic AI suggestion concept	Side-panel review with full apply, partial apply, reject, and version snapshots	Better product integration and closer alignment with the user-story requirements

7.2 Why These Differences Are Defensible

None of these differences are accidental. They are the result of architectural trade-offs made under the assignment's constraints. For example, using SQLite instead of a larger operational database deployment does not weaken the architecture's modularity; it lowers setup friction so the team can spend more effort on requirements, contracts, collaboration correctness, and

testability. Likewise, using local bearer-token authentication instead of full OAuth/OIDC is acceptable in the MVP because the report still explains the role model, trust boundary, and future identity-provider integration direction.

This section exists to make those trade-offs explicit. The architecture remains coherent because the same modules, APIs, and responsibilities described in Part 2 are actually visible in the repository and the demoable system, even when the implementation chooses a coursework-friendly version of a more complex target technology.

Chapter 8

Conclusion

This project demonstrates a coherent architecture for a collaborative editor with integrated AI support, and it validates that architecture through a working MVP rather than through diagrams alone. The final system delivers concrete value across the four assessed areas of the assignment:

- **Requirements engineering:** the report defines functional and non-functional requirements, user stories, and a traceability matrix that links user needs to implemented components.
- **System architecture:** the C4 views, feature decomposition, API contracts, AI integration design, data model, and ADRs explain both the structure and the reasoning behind key design choices.
- **Project management:** ownership boundaries, workflow rules, risk handling, and milestones show how the team could realistically build and evolve the system.
- **Proof of concept:** the repository demonstrates working frontend-backend communication and, in fact, goes beyond the minimum with live collaboration, permissions, AI jobs, autosave, versioning, export, CI, and Docker-based runtime support.

The most important architectural theme throughout the project is deliberate simplification without loss of coherence. Local bearer-token authentication, SQLite, and the stub/LM Studio AI provider strategy are not signs of incomplete thinking; they are examples of choosing the smallest viable implementation that still preserves the essential structure and product behavior required by the assignment. The report therefore treats the delivered MVP honestly: it documents both the intended target architecture and the practical MVP simplifications, and it keeps those two perspectives connected rather than contradictory.

That combination — justified design, clear code structure, and a runnable MVP — is what makes the system a strong foundation for the remainder of the semester.

Appendix A

Appendix

A.1 Submission Artifact Inventory

The final submission package is composed of three coordinated artifacts:

1. **This final report PDF**, which documents requirements, architecture, project management, and proof-of-concept alignment.
2. **The Git repository**, which contains the runnable MVP, root orchestration scripts, OpenAPI/contract notes, tests, Docker Compose configuration, CI workflow, and demo runbook.
3. **The recorded presentation**, which should cover team responsibilities, key design decisions, and the proof-of-concept/demo flow.

A.2 Assignment Coverage Map

Table A.1 maps the assignment brief's required deliverables to the final report and the repository artifacts. This is included deliberately to reduce any ambiguity between the rubric, the written report, and the delivered MVP.

Table A.1: Assignment coverage map

Assignment Requirement	**Covered in Report**	**Covered in Repository / MVP**
Stakeholder analysis	Section 3, Stakeholder Analysis	Reflected in RBAC, AI policy, and deployment decisions
Functional requirements across collaboration, AI, document management, and user management	Section 3, Functional Requirements	Backend routes, frontend flows, realtime service, tests
Measurable non-functional requirements	Section 3, Non-Functional Requirements	Latency/error/reliability constraints reflected in design, scripts, and fallback behavior
At least ten user stories	Section 3, User Stories and Scenarios	Implemented or explicitly scoped in MVP alignment section
Requirements traceability matrix	Section 3, Traceability Matrix	Links stories to repo modules and services

Assignment Requirement	**Covered in Report**	**Covered in Repository / MVP**
Architectural drivers	Section 4, Architectural Drivers	Reflected in modular repo and service boundaries
C4 Level 1, 2, and 3 diagrams	Section 4, Figures and appendix source listings	Mermaid source in repo and final-report package
Feature decomposition	Section 4, Feature Decomposition	Monorepo service layout and domain modules
AI integration design	Section 4, AI Integration Design	AI service abstraction, async jobs, provider selection
API design with concrete contracts	Section 4, API Design	Backend OpenAPI, contract notes, automated tests
Authentication and authorization rationale	Section 4, Authentication and Authorization	Login route, bearer tokens, RBAC, realtime write restrictions
Communication model	Section 4, Communication Model	Backend sessions + Yjs realtime service + frontend reconnect behavior
Code structure and repository organization	Section 4, Code Structure & Repository Organization	Root scripts, monorepo folders, shared code, tests
Data model and ER diagram	Section 4, Data Model	SQLite schema, repositories, versioning, AI jobs, export jobs
Four ADRs	Section 4, Architecture Decision Records	Reflected in code and tooling choices
Team structure and ownership	Section 5, Team Structure and Ownership	PR/branch history and service ownership in repo
Development workflow and methodology	Section 5, Development Workflow and Methodology	Branch structure, CI, root scripts, reviewable commit history
Risk assessment and milestones	Section 5, Risk Assessment and Timeline	Regression tests, demo reset, Docker Compose, CI, milestone table
PoC front-end/back-end communication	Section 6, Proof of Concept	Working MVP and automated smoke tests
README and runnable repository	Section 6, Repository and README Evidence	Root README, service READMEs, DEMO.md, Docker Compose
Editable diagram sources	Appendix Mermaid source listings	‘docs/diagrams/*.mmd‘, ‘final-report/figures/mermaid-src/*.mmd‘

A.3 Version-Control and Collaboration Evidence

The proof-of-concept rubric explicitly evaluates whether the Git history is clean and whether there is evidence of team collaboration. The delivered repository supports that in three ways:

- feature work was kept on named branches and integrated through pull-request-style review rather than as one unstructured dump into `main`,
- commits were intentionally kept small and purposeful so architectural evolution is visible in the history,

- the repository contains both implementation artifacts and editable documentation artifacts so architecture changes can be versioned alongside code rather than maintained separately.

This matters for the assignment because the architecture, repo organization, and PoC are meant to reflect the same design process rather than three unrelated deliverables.

A.4 Demo Checklist

The recommended live demo sequence is:

1. Run `npm run demo:reset` from the repository root.
2. Open the frontend at `http://localhost:5173`.
3. Sign in as `user_1` and create a new document.
4. Open the same document in a second browser/tab and switch the user to `user_2`.
5. Grant editor access from the owner account.
6. Type in both windows and show live sync and presence.
7. Pause briefly to show autosave returning the header to a saved state.
8. Trigger an AI action, wait for the async job, then demonstrate full apply or partial apply.
9. Open version history and demonstrate revert.
10. Export the document and show Swagger/OpenAPI at `http://localhost:3000/docs/`.

A.5 Mermaid Diagram Source Listings

The assignment requires editable diagram source files. The report therefore includes Mermaid source listings for the reconstructed architecture diagrams used as living artifacts.

A.5.1 C4 Level 1 — System Context (Mermaid Source)

```
%%{init: {"theme":"base","flowchart":{"curve":"linear"},"themeVariables":{"background
    ":"#ffffff","primaryColor":"#f8fafc","primaryBorderColor":"#111827","
    primaryTextColor":"#111827","secondaryColor":"#eef2ff","secondaryBorderColor":"#1
    d4ed8","secondaryTextColor":"#111827","tertiaryColor":"#fef3c7","
    tertiaryBorderColor":"#b45309","tertiaryTextColor":"#111827","clusterBkg":"#ffffff
    ","clusterBorder":"#111827","lineColor":"#111827","fontSize":"20px"}}}%%
flowchart TB
    User["User<br/>Uses browser to<br/>edit and collaborate"]
    Admin["Admin<br/>Manages access<br/>and AI permissions"]
    System["System<br/>Real-time collaborative<br/>document editor with<br/>AI
    assistant"]
    IdP["Identity Provider<br/>Authenticates users"]
    LLM["LLM API Provider<br/>External AI service used<br/>for text processing"]

    User -->|edits documents,<br/>triggers AI| System
    Admin -->|manages permissions<br/>and AI limits| System
    System -->|authentication<br/>requests| IdP
    System -->|sends text for<br/>AI processing| LLM
```

A.5.2 C4 Level 2 — Container Diagram (Mermaid Source)

```
%%{init: {"theme":"base","flowchart":{"curve":"linear"},"themeVariables":{"background
    ":"#ffffff","primaryColor":"#f8fafc","primaryBorderColor":"#111827","
    primaryTextColor":"#111827","secondaryColor":"#eef2ff","secondaryBorderColor":"#1
    d4ed8","secondaryTextColor":"#111827","tertiaryColor":"#fef3c7","
    tertiaryBorderColor":"#b45309","tertiaryTextColor":"#111827","clusterBkg":"#ffffff
    ","clusterBorder":"#111827","lineColor":"#111827","fontSize":"20px"}}}%%
flowchart LR
    User["User"]

    subgraph System[Collaborative Editor System]
        direction LR
        RT["Real-Time Service<br/>WebSocket / Yjs relay"]
        FE["Frontend<br/>React Web App"]
        BE["Backend API<br/>Node.js / Express"]
        DB[("Database<br/>SQLite")]
        AIS["AI Integration Service<br/>Job orchestration + provider adapter"]
    end

    IdP["Identity Provider<br/>Target production auth"]
    LLM["LLM API Provider"]

    User --> FE
    FE -->|REST<br/>communication| BE
    FE -->|WebSocket<br/>communication| RT
    BE --> DB
    RT --> DB
    BE --> AIS
    AIS --> LLM
    BE -.->|validate<br/>access token| IdP
```

A.5.3 C4 Level 3 — Backend API Component Diagram (Mermaid Source)

```
flowchart LR
    Client[Web Client / React]
    Router[API Router / Controllers]
    AuthMW[Auth Middleware\nBearer-token validation]
    RBAC[Authorization / RBAC Policy]

    DocSvc[Document Service]
    VersionSvc[Version / Revert Logic]
    PermissionSvc[Permission Service]
    AISvc[AI Job Service]
    ExportSvc[Export Job Service]
    SessionSvc[Session Service]

    UsersRepo[Users Repository]
    DocsRepo[Documents Repository]
    PermRepo[Permissions Repository]
    AIRepo[AI / Policy Repository]
    ExportRepo[Exports Repository]
    Audit[Audit Logger]

    DB[(SQLite Database)]
    RT[Realtime Sync Service\nWebSocket / CRDT Gateway]
    LLM[AI Provider / LLM API]
```

```
    IdP[Identity Provider\nTarget architecture]

    Client -->|REST routes| Router
    Router --> AuthMW --> RBAC

    Router --> DocSvc
    Router --> VersionSvc
    Router --> PermissionSvc
    Router --> AISvc
    Router --> ExportSvc
    Router --> SessionSvc

    DocSvc --> DocsRepo
    DocSvc --> Audit
    VersionSvc --> DocsRepo
    VersionSvc --> Audit
    VersionSvc --> RT
    PermissionSvc --> PermRepo
    PermissionSvc --> Audit
    PermissionSvc --> RT
    AISvc --> AIRepo
    AISvc --> Audit
    AISvc --> LLM
    ExportSvc --> ExportRepo
    ExportSvc --> Audit
    SessionSvc --> DocsRepo
    SessionSvc --> PermRepo
    SessionSvc --> UsersRepo

    UsersRepo --> DB
    DocsRepo --> DB
    PermRepo --> DB
    AIRepo --> DB
    ExportRepo --> DB
    Audit --> DB
    AuthMW -.-> IdP
```

A.5.4 Entity-Relationship Diagram (Mermaid Source)

```
erDiagram
    USERS ||--o{ DOCUMENTS : owns
    USERS ||--o{ DOCUMENT_PERMISSIONS : receives
    DOCUMENTS ||--o{ DOCUMENT_PERMISSIONS : grants
    DOCUMENTS ||--o{ DOCUMENT_VERSIONS : snapshots
    DOCUMENTS ||--|| AI_POLICIES : has
    DOCUMENTS ||--o{ AI_JOBS : scopes
    DOCUMENTS ||--o{ EXPORT_JOBS : scopes
    DOCUMENTS ||--o{ REALTIME_EVENTS : emits
    DOCUMENTS ||--o{ AUDIT_LOGS : records
    USERS ||--o{ AI_JOBS : creates
    USERS ||--o{ EXPORT_JOBS : creates
    USERS ||--o{ AUDIT_LOGS : performs
    DOCUMENTS ||--o{ IDEMPOTENCY_KEYS : guards
    DOCUMENT_VERSIONS ||--o{ AI_JOBS : base_version

    USERS {
      text user_id PK
```

```
  text display_name
  text global_role
  text access_token
  text created_at
  text updated_at
}

DOCUMENTS {
  text document_id PK
  text owner_user_id FK
  text title
  text content
  text created_at
  text updated_at
  text current_version_id FK
  text revision_id
}

DOCUMENT_PERMISSIONS {
  text document_id FK
  text user_id FK
  text role
  text updated_at
}

DOCUMENT_VERSIONS {
  text version_id PK
  text document_id FK
  int version_number
  text created_at
  text created_by_user_id FK
  text reason
  text snapshot_content
  text base_revision_id
}

AI_POLICIES {
  text document_id PK
  int ai_enabled
  text allowed_roles_csv
  int daily_quota
  text updated_at
}

AI_JOBS {
  text job_id PK
  text document_id FK
  text user_id FK
  text action_type
  text status
  text request_json
  text result_json
  text error_code
  text error_message
  text base_version_id FK
  text created_at
  text updated_at
}
```

```
EXPORT_JOBS {
  text job_id PK
  text document_id FK
  text user_id FK
  text format
  text status
  text result_json
  text created_at
  text updated_at
}

REALTIME_EVENTS {
  text event_id PK
  text document_id FK
  text event_type
  text payload_json
  text created_at
  text delivered_at
}

IDEMPOTENCY_KEYS {
  text scope PK
  text request_id PK
  int response_status
  text response_body
  text created_at
}

AUDIT_LOGS {
  text event_id PK
  text actor_user_id FK
  text document_id FK
  text action_type
  text metadata_json
  text created_at
}
```

A.6 Additional Mermaid Source Artifacts Available in the Repository

The repository also includes supplementary Mermaid sources for dynamic interaction views:

- `final-report/figures/mermaid-src/sequence-document-open-and-sync.mmd`
- `final-report/figures/mermaid-src/sequence-ai-job-flow.mmd`

These are maintained as editable assets but are intentionally not expanded into full code listings in the main report PDF, because the assignment's core diagram requirement is already satisfied through the C4 and ER views included above.